

Software Architecture Document

V.A.R.I.O.

Voice Adaptive Role & Intent Orchestrator

Team 05 PIPELINE (Project ID 42)

Rohith G S · Kurumeti Mitesh · Mohammed Tahir Siddique · Pavan Kishor M

UE24CS341A — Software Engineering, PES University

Table of Contents

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms and Abbreviations
 - 1.4 References
2. Architectural Representation
3. Architectural Goals and Constraints
4. Use-Case View
 - 4.1 Architecturally-Significant Use Cases
5. Logical View
 - 5.1 Architecture Overview – Package and Subsystem Layering
Module responsibilities
6. Process View
 - 6.1 Processes
 - 6.2 Process to Design Elements
 - 6.3 Process Model to Design
 - 6.4 Model Dependencies
 - 6.5 Processes to the Implementation
7. Deployment View
 - 7.1 Client Device (End User)
 - 7.2 Client Device (Role Admin)
 - 7.3 Application Server Node (OCI Ampere A1)
 - 7.4 Mock Systems Node (Integration Layer)
 - 7.5 Data & External Services Node
8. Performance
9. Quality
 - Reliability
 - Security
 - Safety
 - Maintainability & Extensibility
 - Usability
 - Testability

1. Introduction

1.1 Purpose

This document is the Software Architecture Document (SAD) for V.A.R.I.O., a role-specific, context-aware conversational assistant framework. It describes the architecture using a set of views derived from the “4+1” model — Use-Case, Logical, Process, and Deployment — so that developers, reviewers, and the course evaluator have a single, consistent picture of how the system is built and why.

The intended audience is the development team (all four members), the course instructor/evaluator, and any future maintainer who needs to extend the system with a new role pack or swap a mock integration for a real one.

1.2 Scope

This document covers the architecture of the V.A.R.I.O. platform as scoped for v1:

- The shared core conversational engine (session management, dialogue routing, response rendering).
- Three pluggable role packs (HR, IT Support, Admissions), each built on an isolated Rasa NLU instance.
- The integration-adapter layer and its four mock backend services (Auth, HRMS, ITSM, Admissions).
- Text and voice (talk mode) interaction channels.
- The role-scoped admin configuration console (intents, FAQs, templates, retraining, audit log, conversation logs).
- The deployment topology (OCI, Cloudflare Tunnel, Vercel, Neon) and CI/CD pipeline.

Out of scope: real enterprise system integrations, multi-language support, native mobile apps, advanced analytics/BI, and proactive/push notifications. These are documented in docs/PRD.md and are not addressed further here.

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
VARIO	Voice Adaptive Role & Intent Orchestrator — the system itself
Role Pack	A pluggable, department-specific configuration (vocabulary, intents, workflows) — HR, IT, or Admissions
NLU	Natural Language Understanding
Rasa	Rasa Open Source — the NLU/dialogue-management framework used for each role pack
Adapter	A component implementing a common interface in front of one backend system (real or mock)
HRMS / ITSM	Human Resource Management System / IT Service Management system

Term	Meaning
Mock service	A FastAPI stand-in simulating a real departmental backend during development/demo
FR / NFR	Functional Requirement / Non-Functional Requirement
JWT	JSON Web Token — used for VARIO's own session authorization after delegated auth
Slot	A named piece of information collected across conversation turns (e.g., leave_type)
Intent	A classified user goal (e.g., check_leave_balance)
dynamic_workflow	An intent type pre-seeded by developers; calls a real backend; name/variables locked
static_faq	An intent type created and fully owned by a role admin; no backend call
Chat mode / Talk mode	Text-only interaction / voice-in, voice-out interaction over the same pipeline

1.4 References

Document	Content
docs/PRD.md	Product requirements, user stories, success criteria
docs/vario-overview.md	Single-page project briefing
docs/ADD.md	Architecture & Design Document — stack, modules, data model, flows
docs/api-contract.md	Frozen REST API contract (envelope, endpoints, error codes, rate limits)
docs/ui-reference.md	UI page inventory, bot message types, states
docs/research.md	Problem discovery and resolved open questions
docs/decisions/ADR-001-delegated-auth.md	Decision record: delegated authentication model
docs/sprints/*.md	Sprint plans and per-member lanes
docs/slices/*.md	Individual work-unit specifications with acceptance criteria
AGENTS.md	Engineering process rules for contributors (human/agent)

2. Architectural Representation

The architecture is represented using four views, plus the use cases that motivate them (the “4+1” approach):

- Use-Case View — the architecturally significant scenarios that drove structural decisions.
- Logical View — the system decomposed into modules/subsystems and their responsibilities and layering.
- Process View — the runtime processes/services, how requests flow through them, and how they map to the module design.
- Deployment View — the physical nodes the system runs on and how they connect.

Each view is documented with enough detail to explain why a decision was made, not only what was built, because several decisions (delegated auth, three isolated Rasa instances, adapter-per-backend) deliberately diverge from the most obvious “simplest” design in favor of the extensibility and reliability goals stated in Section 3.

3. Architectural Goals and Constraints

Goals (drawn from docs/PRD.md success criteria and docs/vario-overview.md NFRs):

- Extensibility — a new role pack must be addable via configuration + one new Rasa instance + one new adapter, with no core-engine code changes, in under one developer-day.
- Reusability — one core engine (session, routing, rendering) shared by all role packs, eliminating the duplicated-engineering problem identified in docs/research.md.
- Swappability — every backend integration sits behind a common adapter interface so a mock can be replaced by a real HRMS/ITSM/Admissions system without touching Rasa or the core engine.
- Safety — no record-affecting action (leave submission, ticket creation, password reset) executes without an explicit user confirmation step, and every such action is audit-logged.
- Graceful degradation — a downed mock service or exhausted TTS quota must inform the user, never crash the pipeline.
- Continuous delivery — the full CI/CD pipeline is stood up in Sprint 1 against a trivial skeleton so every subsequent merge auto-deploys; there is no big-bang integration at the end.

Constraints:

- Team size / timeline: 4 developers, 7 weeks, alongside coursework — scope is deliberately tight (English only, web only, mocks only, no push notifications).
- Performance: 95% of queries respond within 2 seconds, excluding artificial mock-API delay.
- Scale: designed for 20 concurrent demo sessions, not production traffic — infra choices (OCI/Neon/Gemini free tiers) reflect this.
- Security: JWT required on all protected routes; admin routes additionally scoped by role claim; secrets only via environment variables / GitHub Actions secrets, never committed or hardcoded.

- Auth model: VARIO does not manage user accounts, passwords, or resets — authentication is fully delegated to the organization's identity provider (ADR-001), simulated by a Mock Auth Service in dev/demo.
- Frozen contract: docs/api-contract.md is frozen for the project's duration so the three role-pack owners can build against it in isolation before integrating with the core engine.

4. Use-Case View

4.1 Architecturally-Significant Use Cases

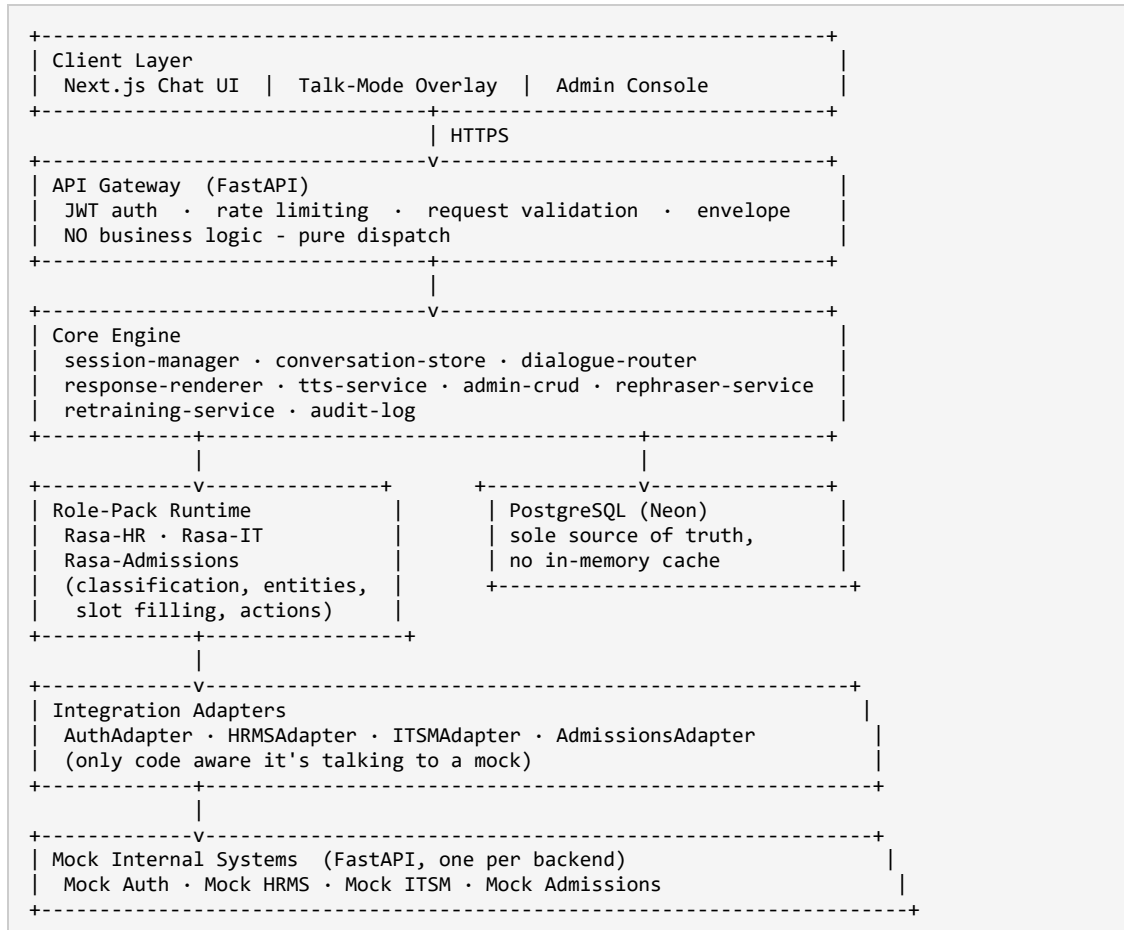
These use cases were chosen because each one forced a specific structural decision, rather than being satisfiable by any arbitrary implementation.

ID	Actor	Summary	Architectural impact
UC-1	Employee	Ask leave balance in natural language → HRMS adapter call → balance returned	Established the core chat pipeline: Gateway → Session Manager → Dialogue Router → Rasa → Adapter → Response Renderer. Also proved the adapter/mock swap seam.
UC-2	Employee	State a leave-request intent → multi-turn slot fill → confirmation → submission → reference ID	Drove the session/slot-persistence design (Postgres as sole source of truth) and the mandatory confirmation-before-write safety rule.
UC-3	Employee	Describe an IT issue → ticket created via ITSM adapter, with duplicate suppression on an immediate repeat	Required session-scoped state beyond simple slots (a request signature) to satisfy FR-13.
UC-4	Employee	Ask ticket status by ID → adapter query → status or invalid-ID guidance	Established the “not-found” bot message type and graceful-degradation contract for adapters.
UC-5	Applicant	Ask application status by ID → Admissions adapter query → status + suggested next step	Required adapters to return structured “not found + guidance” results rather than throwing.
UC-6	Role Admin	Log in with a role-scoped JWT → add/edit/remove an intent or FAQ → change goes live with no redeploy	Drove the separation between <code>dynamic_workflow</code> (developer-owned, locked name/variables) and <code>static_faq</code> (fully admin-owned) intent types, and the manual-retrain-as-a-service design (FR-3).
—	Employee (voice)	Speak to the bot via microphone and hear the response spoken back	Drove the decision to do STT in-browser (Web Speech API) and TTS server-side (Google Cloud), with the same chat pipeline underneath — talk mode is a thin extension of chat mode, not a parallel system.

5. Logical View

5.1 Architecture Overview – Package and Subsystem Layering

The system is layered so that no layer needs to know the internal details of the layer two levels away — in particular, Role Packs never talk to Mock Systems directly; they always go through an Adapter.



Module responsibilities

Module	Responsibility
gateway	Single HTTP endpoint; envelope, JWT middleware, rate limiting, schema validation
auth	Forwards credentials to Mock Auth / org IdP; issues VARIO's JWT with a role claim
session-manager	Loads/persists current_form, current_step, slots, expires_at per conversation
conversation-store	Durable conversation + message history, for both end-user and admin views

Module	Responsibility
dialogue-router	Reads role_pack, forwards to the matching Rasa instance; no NLU logic itself
role-pack-runtime	Three isolated Rasa instances — classification, entities, slot filling, custom actions
integration-adapters	One per backend system; common interface; only place that knows it's a mock
response-renderer	template_key + slots → random response_variants row → placeholder substitution
tts-service	Google Cloud TTS wrapper for mode:"talk"; returns a degrade flag on failure
admin-crud	Role-scoped CRUD for intents/phrases/templates/variants; sets needs_retrain
rephraser-service	LLM-backed (Gemini, swappable) generation of response variants and phrase suggestions
retraining-service	Role pack's DB data → Rasa YAML → rasa train → reload → clears needs_retrain
audit-log	Fire-and-forget sink for business-significant events

Data model (9 entities): users, conversations, messages, sessions, intents, training_phrases, response_templates, response_variants, audit_log. Full field-level detail is in docs/ADD.md § Data model. Notably there is no auth_tokens table — a direct consequence of the delegated-authentication decision (ADR-001).

6. Process View

6.1 Processes

The system is deployed as a set of independently runnable services (processes), reflecting the module boundaries above:

1. Frontend process — Next.js app (chat UI, talk-mode overlay, admin console).
2. Gateway/Core Engine process — FastAPI backend hosting the gateway, session manager, dialogue router, response renderer, admin-crud, rephraser-service, retraining-service, tts-service, and audit-log. These run in one FastAPI process for simplicity at this scale.
3. Rasa-HR process — isolated NLU/dialogue-management server for the HR role pack.
4. Rasa-IT process — isolated NLU/dialogue-management server for the IT role pack.
5. Rasa-Admissions process — isolated NLU/dialogue-management server for the Admissions role pack.
6. Mock Auth process — simulates the org identity provider.
7. Mock HRMS process — simulates the HR system of record.
8. Mock ITSM process — simulates the IT service-management system.
9. Mock Admissions process — simulates the admissions system.
10. PostgreSQL process — the Neon-hosted database, external to the application processes.

Each Rasa and mock process is isolated on purpose: a crash or slow response in one role pack (e.g., Rasa-IT) must not affect another (e.g., Rasa-HR), and each mock can be independently restarted without redeploying the core engine.

6.2 Process to Design Elements

Process	Design elements (modules) it hosts
Gateway/Core Engine	gateway, session-manager, conversation-store, dialogue-router, response-renderer, tts-service, admin-crud, rephraser-service, retraining-service, audit-log
Rasa-HR / Rasa-IT / Rasa-Admissions	one instance each of role-pack-runtime, plus that role pack's custom actions calling its integration-adapters
Mock Auth / HRMS / ITSM / Admissions	standalone FastAPI services with no shared code with the core engine — only the corresponding adapter knows their shape
Frontend	Chat View shell, shared message component library, Talk Mode overlay, Admin Console framework and pages

6.3 Process Model to Design

The chat-mode request/response cycle is the canonical process interaction and is used to validate that the design elements above compose correctly at runtime:

```
User -> Frontend -> Gateway (JWT check)
```

```

-> Session Manager (load state, persist user message)
-> Dialogue Router (reads conversation.role_pack)
-> Rasa-<RolePack> (classify intent + entities)
    -> Integration Adapter -> Mock System (if a workflow needs data)
-> Rasa returns {template_key, slots, intent, confidence}
-> Response Renderer (random variant + substitution)
-> Session Manager (persist bot message, update session)
-> Audit Log (fire-and-forget, if a business event occurred)
-> Gateway -> Frontend (renders the bot message type)

```

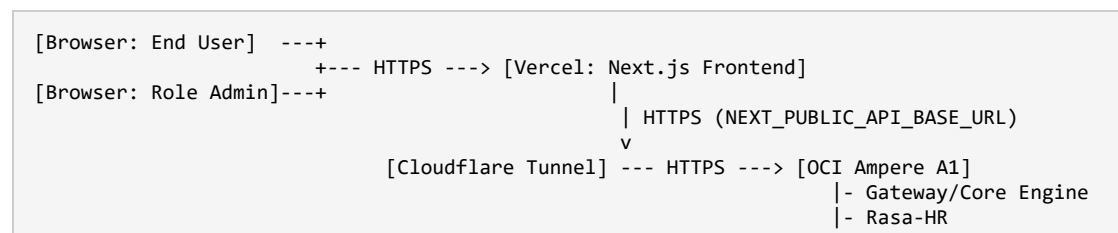
Below-confidence-threshold classification short-circuits this flow into a clarification prompt rather than reaching a workflow (FR-2). Talk mode appends a single step after rendering: the Gateway calls tts-service, which returns audio bytes or a tts_available:false flag, never blocking the text response.

6.4 Model Dependencies

- dialogue-router depends on session-manager having already loaded/persisted state for the turn — it must not run against stale session data.
- role-pack-runtime depends on integration-adapters for any workflow requiring live data; it never depends on a specific mock's schema.
- response-renderer depends on response_templates / response_variants existing in Postgres before a role pack is usable — retraining and admin CRUD are therefore upstream dependencies of a functioning chat flow.
- admin-crud depends on the role-based authorization guard (built once) so that A/B/C's role-scoped admin pages cannot cross role-pack boundaries.
- retraining-service depends on admin-crud's data (intents + phrases) and on each Rasa instance exposing a reload mechanism.
- tts-service is a leaf dependency — nothing else depends on it, and its failure must not cascade.

6.5 Processes to the Implementation

Process	Implementation technology	Notes
Frontend	Next.js (React), deployed to Vercel	Talks to the backend only via NEXT_PUBLIC_API_BASE_URL
Gateway/Core Engine	FastAPI (Python), containerized, deployed to OCI	Single container in v1; modules are logically separated in code, not physically split
Rasa-HR / IT / Admissions	Rasa Open Source, containerized, deployed to OCI	Reachable at RASA_HR_URL / RASA_IT_URL / RASA_ADMISSIONS_URL
Mock Auth / HRMS / ITSM / Admissions	FastAPI (Python), containerized, deployed to OCI	Reachable at MOCK_*_URL; swappable for real systems later with no interface change
Database	PostgreSQL via Neon	Managed, serverless; dev and prod databases/branches
Local development	Docker Compose	Brings up all 10 processes with one command





8. Performance

- Target: 95% of queries answered within 2 seconds, excluding artificial mock-API delay deliberately built into the mocks for testing the performance carve-out.
- Talk mode overhead: TTS synthesis adds roughly 1–2 seconds on top of the text response time; this is budgeted separately and is not part of the 2-second chat target.
- Admin operations: template save with synchronous variant generation takes ~3–5 seconds (LLM round-trip) and is explicitly exempted from the interactive-chat performance target since it is an infrequent admin action, not an end-user-facing hot path.
- Retraining: long-running (~30–120 seconds), run as an asynchronous job with client-side polling (GET /api/admin/retrain/:job_id) rather than a blocking request.
- Scalability target: the system must support 20 concurrent user sessions during a live demo with no noticeable degradation. This is validated by a dedicated load-test harness run early in Sprint 4, with results recorded in docs/perf-report.md.
- Rate limiting at the Gateway protects both performance and fairness: /api/chat 30/min/user, /api/conversations/* 60/min/user, /api/admin/* 30/min/user, /api/admin/retrain 3/hour/user, /api/auth/* 10/min/IP.
- Isolating each role pack into its own Rasa process means a slow or heavily-loaded HR conversation does not degrade IT or Admissions response times.

9. Quality

Reliability

- Every integration adapter returns typed results (not-found, backend-unavailable) rather than letting exceptions propagate — a downed mock informs the user gracefully instead of crashing the pipeline.
- Session state lives entirely in PostgreSQL (no in-memory cache), so a conversation can resume correctly after a transient backend restart.
- TTS failures degrade to text-only (tts_available:false) without blocking the underlying text response.

Security

- All protected routes require a valid VARIO-issued JWT; admin routes are additionally scoped by the role claim, so an HR admin cannot read IT or Admissions data.
- Authentication is fully delegated (ADR-001) — VARIO never stores passwords, handles registration, or manages lockouts; that responsibility stays with the organization's identity provider.
- Secrets (JWT signing key, Gemini API key, Google Cloud TTS credentials, database URL) are read from environment variables / GitHub Actions secrets only, and are explicitly excluded from version control.
- Input is validated/sanitized at the gateway before it reaches NLU or persistence.

Safety

- No record-affecting workflow (leave submission, ticket creation, password reset) executes without an explicit user confirmation step.
- Every such action produces an audit_log entry (actor_id, role_pack, action_type, reference_id, details).

Maintainability & Extensibility

- Core Engine, each Role Pack, and the Integration Adapter layer are independently testable services with clearly owned interfaces.
- Adding a new role pack requires only a new Rasa instance, a new adapter, and configuration/seed data — no core-engine code changes.
- The rephraser is built behind a swappable RephraserClient interface (passthrough default, then real Gemini-backed generation), so CRUD functionality was never blocked on the LLM integration landing.

Usability

- Both text and voice channels are supported through the identical chat pipeline; when the browser lacks the Web Speech API, text chat remains fully usable — voice is additive, never a hard requirement.
- Bot responses avoid jargon in error/fallback states and use consistent, reusable message components (data card, slot prompt, confirmation, result, not-found, FAQ answer, error, list) across all three role packs.

Testability

- Every functional requirement (FR-1 through FR-19) has at least one associated acceptance criterion in its owning slice, treated as the test contract.
- Backend tests run via pytest, frontend tests via Jest/React Testing Library, gated in CI on every pull request before merge.